

fumat

Architecture, decisions, and change log · started 2026-09-30

This is the single developer document for fumat. Compile it from the repository root with `typst compile --root . notes/fumat.typ`; the chapters live in `notes/architecture.typ` and are not compiled on their own. The layout follows `element-driver/notes`, and `template.typ` is a copy of the one there.

- Section 1 covers where fumat sits among the CriticalSoilModels repositories, its decisions (F1, F2, ...), the proposed module layout, the open questions, and the milestones.

Decisions of `element-driver` are cited as D1, D2, ... (see `element-driver/notes/design.typ`).

Tags. **RESOLVED** decided; **PROPOSED** designed but not in code, awaiting agreement; **OPEN** not yet decided; **DEFERRED** agreed to leave for later; **TECH DEBT** style or structure, not correctness.

Contents

1	Architecture of fumat	2
1.1	Purpose	2
1.2	Place among the repositories	2
1.3	Decisions	2
1.4	<code>csm-tensors</code>	3
1.5	What goes in <code>record_t</code>	4
1.6	Quad plot	5
1.7	Proposed module layout	5
1.8	Calibration	5
1.9	Open questions	6
1.10	Current state (review of 2026-09-30)	6
1.11	Milestones	7
1.12	Change log	7

1 Architecture of `fumat`

1.1 Purpose

`fumat` post-processes the results of constitutive-model element tests. It takes stress, strain, and time histories, from any solver or from the lab, derives the usual soil-mechanics quantities, plots them, and compares model results with measurements. The comparison pieces are the building blocks of calibration.

It is written for single-element results first, starting with `element-driver` (F9), but nothing in it knows which solver produced the data. A material-point history from an FEM or MPM code (e.g. `GeoMPM.jl`) is used the same way.

1.2 Place among the repositories

Repository	Role	Depends on
<code>csm-tensors</code>	Voigt conventions, stress and strain invariants and their derivatives, stress spaces, elastic conversions, the <code>wp</code> kind. Pure functions, no state (Section 1.4).	<code>stdlib</code>
<code>critical-soil-models</code>	Soil constitutive models, solver independent.	<code>stdlib</code> , <code>csm-tensors</code>
<code>fumat</code>	Post-processing, plotting, model-data comparison. Solver independent (F1).	<code>stdlib</code> , <code>fortplot</code> , <code>csm-tensors</code>
<code>element-driver</code>	Single-element solver. A library with no files or printing (D4).	<code>critical-soil-models</code> , <code>fumat</code> (F8), <code>stdlib</code> , <code>csm-tensors</code>
<code>Incremental_Driver</code>	Frozen legacy driver.	–

The dependency arrows point one way. `csm-tensors` is the common leaf. `critical-soil-models` and `fumat` sit beside each other and do not know about each other, and `element-driver` sits on top. `fumat` never imports a solver or a model. It learns about results only through arrays (F2).

Change from the earlier plan

`Notes/architecture_decisions.md` (parent folder) had `fumat` on top of the driver and calling it in memory for calibration. With F1 the direction is reversed: the driver depends on `fumat`, and calibration calls the solver through an interface that `fumat` defines and the solver side implements (Section 1.8). That note should be updated to point here.

1.3 Decisions

Discussed 2026-09-30.

#	Decision	Status
F1	<code>fumat</code> is solver independent and depends on no solver or model repository. <code>element-driver</code> depends on <code>fumat</code> , not the other way round. <code>fumat</code> must be usable with any solver that can hand over arrays or write a file.	RESOLVED
F2	Inputs are plain arrays or files. Every analysis works on arrays in memory; file readers (later, F9) only fill those arrays.	RESOLVED
F3	Data are stored with the sign convention of <code>critical-soil-models</code> and <code>element-driver</code> (D16): tension positive, compression negative, for stress and strain. Plots show the data as stored by default, so compression is negative. An optional flag (<code>compression_positive = .true.</code>) flips the plotted quantities to the geotechnical convention. The flag only affects what is drawn; the data are never changed in place.	RESOLVED

#	Decision	Status
F4	Calibration pieces are built in <code>fumat</code> first: putting model and data on a common axis, and residuals. Where the optimizer loop lives (<code>fumat</code> or a separate repository) is decided once those pieces exist (Q6).	RESOLVED
F5	Voigt order (11, 22, 33, 12, 13, 23) with engineering shear strain $\gamma_{ij} = 2\varepsilon_{ij}$, as in <code>critical-soil-models</code> and the original driver. Confirmed 2026-09-30: <code>calc_two_norm_tensor_strain</code> halves the shear terms, and <code>element-driver</code> 's Roscoe strain rows are work conjugate with it. Now one definition, in <code>csm-tensors</code> .	RESOLVED
F6	Same library style as <code>element-driver</code> (D4, D11): no global state, no stop in library code, errors returned as a <code>stdlib_error: state_type</code> . Plotting and file writing are the only side effects, and only in the routines named for them.	PROPOSED
F7	Toolchain aligned with <code>element-driver</code> : gfortran only (D13), <code>stdlib</code> at <code>.././stdlib</code> , <code>fortplot</code> from <code>lazy-fortran/fortplot</code> , <code>test-drive</code> for tests, and notes in <code>Typst</code> with the compile hook.	PROPOSED
F8	<code>fumat</code> is a regular dependency of <code>element-driver</code> (was Q1). Its examples plot with <code>fumat</code> . Downstream programs also build <code>fortplot</code> , which is small and builds easily.	RESOLVED
F9	<code>element-driver</code> is the first data source, handed over in memory. Text files (CSV output, legacy <code>Incremental_Driver</code> files, lab data) are left for later (was Q4, Q5).	RESOLVED
F10	Invariants and Voigt helpers are implemented once, in <code>csm-tensors</code> , which <code>critical-soil-models</code> , <code>element-driver</code> , and <code>fumat</code> all depend on (was Q3; Section 1.4). The name leaves room for more tensor code later.	RESOLVED
F11	No <code>record_t</code> for now (was Q2, option A of Section 1.5). Plot objects take arrays one test at a time. A record type shaped like <code>element-driver</code> 's <code>history_t</code> would overfit <code>fumat</code> to one solver's output format.	RESOLVED
F12	Quad plot (<code>quad_plot_t</code> , Section 1.6): q vs ε_q , q vs p , ε_v vs ε_q , ε_v vs p on a 2×2 grid, with the invariant pair $q = \sqrt{3J_2}$, $\varepsilon_q = \sqrt{2/3} \mathbf{e} : \mathbf{e}$ (changed from the signed triaxial pair on 2026-09-30: frame independent and defined for any path). ε_q is the default strain axis; axial strain is an option for lab data ($\varepsilon_a = \varepsilon_q + \varepsilon_v/3$ in triaxial compression, taken compression positive; equal only without volume change).	RESOLVED

1.4 csm-tensors

RESOLVED Created 2026-09-30 (milestone 0) at `../csm-tensors`; not yet on GitHub.

From	Moved into <code>csm-tensors</code>
<code>critical-soil-models</code>	<code>mod_csm_kinds</code> (wp), <code>src/invariants/</code> (values and derivatives), <code>mod_voigt_utils</code> , <code>mod_voigt_conventions</code> , <code>mod_elastic_utils</code> , and the 13 invariant tests with their reference implementations. Module names unchanged, so no use statement in <code>critical-soil-models</code> changed.
<code>element-driver</code>	<code>mod_ed_stress_spaces</code> , renamed <code>mod_stress_spaces</code> , with its 2 tests. Its duplicates of <code>calc_p_inv</code> and <code>calc_eps_vol_inv</code> were dropped in favour of the <code>critical-soil-models</code> ones (same formulas). <code>mod_ed_kinds</code> now takes wp from <code>mod_csm_kinds</code> .
new	<code>mod_invariant_histories</code> : <code>calc_p_history(sig)</code> , <code>calc_q_history</code> , <code>calc_lode_history</code> , <code>calc_eps_vol_history</code> , <code>calc_eps_q_history</code> , <code>calc_q triax_history</code> , <code>calc_eps_q triax_history</code> , each for <code>sig(6, n)</code> or <code>eps(6, n)</code> , with 3 tests.

Results: csm-tensors 18 tests, critical-soil-models 34 (was 47), element-driver 34 (was 36), all passing. The element-driver example plots are byte-identical to those from before the move. The two meanings of q keep separate names: `calc_q_inv` ($\sqrt{3J_2} \geq 0$) and `calc_q_triax` (signed).

Elastic conversions in csm-tensors. Moved, as agreed.

- For: they are general elasticity, not soil models. `fumat` needs them without depending on `critical-soil-models` (F1), for example to draw the elastic slope $3G$ on a q - ε_q plot or to seed calibration from E and ν . element-driver's linear elastic model could use them too.
- Against: the name says tensors, and $K(G, \nu)$ is scalar algebra, so the scope is less sharp. Elasticity has open-ended scope (anisotropic, pressure-dependent $G(p)$), and those models belong in `critical-soil-models`. The line to hold: csm-tensors has isotropic conversions and the constant stiffness matrix only.

Not done yet. The test helpers `tensor_value_check.f90` and `tensor_NaN_check.f90` are now copied in both `critical-soil-models/test` and `csm-tensors/test`. The move did not keep git history; the old files are in `critical-soil-models` before this change and `element-driver` commit `0cbb300`.

1.5 What goes in record_t

RESOLVED Option A chosen (F11). With F9 there are no file readers yet, so the question is only what `fumat` needs to plot and compare `element-driver` results.

What one test consists of.

Field	Need	Notes
<code>sig(6, n), eps(6, n)</code>	required	Everything plotted is derived from these (with the invariants library).
<code>time(n)</code>	usually	Creep and relaxation plots, rate-dependent models. <code>element-driver</code> has <code>t_total</code> .
<code>label</code>	for plots	Legend entry, e.g. " <code>p0 = 100 kPa</code> ".
<code>step(n)</code>	useful	Select one stage, e.g. only the shearing after consolidation. <code>element-driver</code> has <code>step</code> and <code>increment</code> .
state variables	later	Plastic strain, hardening variables, void ratio. Model specific, so they would need names.
measured extras	later	Pore pressure, void ratio, cell pressure; only from lab files (F9).

`element-driver`'s `history_t` also has `n_iter` (Newton iterations), which is solver diagnostics rather than test data.

Option A: no record type yet; the figure holds the data. Plot objects take arrays one test at a time and keep what they need:

```
type(triaxial_plot_t) :: fig
do i = 1, 3
  call fig%add(history(i)%sig, history(i)%eps, label=labels(i))
end do
call fig%save("output/triaxial_mcsc.png", compression_positive=.true.)
```

This keeps the whole API array based (F2), and `element-driver`'s core types stay free of `fumat` types. The cost: each analysis routine repeats the argument list (`sig`, `eps`, `time`, ...), and a record type still arrives with the file readers.

Option B: a small record_t now. `label`, `time`, `sig`, `eps`, and optional `step`, built from arrays by a constructor. Plots and comparisons take `record_t`. Variant B2: `element-driver`'s `history_t` extends `record_t` (adding `increment`, `n_iter`), so a history is passed to `fumat` with no copy and no conversion. B2 ties `element-driver`'s core result type to `fumat`, so a change to `record_t` changes the driver's API.

Decision: option A (F11). A record type is revisited only with the file readers, and then shaped by the files, not by `history_t`.

1.6 Quad plot

RESOLVED In code 2026-09-30: `src/fm_quad_plot.f90`, re-exported by use `fumat`.

```
type(quad_plot_t) :: fig
do i = 1, 3
  call fig%add(history(i)%sig, history(i)%eps, label=labels(i))
end do
fig%compression_positive = .true.      ! F3; default .false.
call fig%save("output/triaxial_mcscs.png", status)
```

- **Variables.** Invariants only, from `csm-tensors (mod_invariant_histories)`: $p = \text{tr}(\sigma)/3$, $q = \sqrt{3J_2} \geq 0$, $\varepsilon_v = \text{tr}(\varepsilon)$, $\varepsilon_q = \sqrt{2/3} \mathbf{e} : \mathbf{e} \geq 0$ (shear terms halved, F5). They do not depend on the axes of the test, so the plot also works for simple shear or true triaxial paths. $q d\varepsilon_q$ equals the deviatoric work $\mathbf{s} : d\mathbf{e}$ when the strain increment is coaxial with $\mathbf{s}$, as in any triaxial test; in general it does not. In a triaxial test $q = |\sigma_a - \sigma_r|$ and $\varepsilon_q = \frac{2}{3} |\varepsilon_a - \varepsilon_r|$, the sizes of the signed Roscoe variables. The signed pair (`calc_q_triax`, `calc_eps_q_triax`) stays in `csm-tensors` for `element-driver`'s constraint rows but is not plotted.
- **Signs.** q and ε_q are magnitudes and never flipped; `compression_positive` flips p , ε_v , and the optional axial strain. Triaxial compression and extension therefore look the same in q ; the Lode angle tells them apart (not plotted).
- **Options** (components, set before save): `compression_positive`, `strain_in_percent` (default true), `use_axial_strain`, `title`, `stress_unit` (default kPa), `width`, `height`.
- **Errors.** `add` has no status; the first shape error is kept and returned by `save` as a `state_type` (F6).
- **Legend workaround (Q7).** Each test gets a fixed colour from the matplotlib `tab10` cycle, and the colour key (blue: 50 kPa orange: 100 kPa ...) is the title of the top-left panel. `suptitle` is one line only, and a long key is clipped at the panel width, so labels should be short.
- **Tests.** 5 in `test/test_fm_quad_plot_suite.f90` (errors, PNG written with default and all options). The plots themselves were checked by eye: `example/quad_plot_synthetic.f90` and the `element-driver` examples `triaxial_linear_elastic` (matches $q = E\varepsilon_a$, $\varepsilon_v = (1 - 2\nu)\varepsilon_a$) and `triaxial_mcscs`.

1.7 Proposed module layout

PROPOSED Names follow `element-driver` (D14): a `wp` kind, `sig` and `eps` as $(6, n)$ arrays, and a `mod_fm_` prefix for modules, like `mod_ed_` there.

Module	Contents
<code>mod_fm_quad_plot</code>	RESOLVED The triaxial quad plot (Section 1.6).
<code>mod_fm_kinds</code>	RESOLVED <code>wp</code> from <code>csm-tensors</code> .
more plots	Oedometer ($\sigma_v - \varepsilon_a$, $\sigma_h - \sigma_v$), a general x - y overlay, following the pattern of <code>quad_plot_t</code> .
<code>mod_fm_compare</code>	DEFERRED Model-data comparison: interpolate the model onto the measured abscissa (e.g. axial strain), residual vectors, weights, and normalization per quantity.
<code>mod_fm_record</code> , <code>mod_fm_csv</code> , <code>mod_fm_lab</code>	DEFERRED With the text files (F9).

Invariants come from `csm-tensors (mod_invariant_histories)`, not from a `fumat` module. `element-driver`'s examples then call `mod_fm_plot` instead of carrying their own `p_of`, `q_of`, `eps_v_of` helpers and sign flips.

1.8 Calibration

DEFERRED Recorded so that F1 does not block it later.

A calibration loop needs to run the solver many times, but by F1 `fumat` cannot call `element-driver`. The usual way round this is an abstract type in `fumat`, extended on the solver side:

```

type, abstract :: simulator_t
contains
  procedure(simulate_iface), deferred :: simulate
    ! simulate(self, params, results, status): run every test for one parameter set
end type

```

The optimizer (e.g. modern-minpack's Levenberg–Marquardt, as planned in Notes/architecture_decisions.md) sees only `simulator_t`, the measured data, and `mod_fm_compare`. An element-driver simulator lives in `element-driver` or in the user's program. Because `element-driver` has no global state (D11), parameter sets can run in parallel.

1.9 Open questions

#	Question	Status
Q1	Regular or dev-dependency? Regular (F8).	RESOLVED
Q2	What goes in <code>record_t</code> ? Not needed now (F11).	RESOLVED
Q3	Shared invariants library? <code>csm-tensors</code> (F10).	RESOLVED
Q4	CSV reader (<code>stdlib_io</code> : <code>loadtxt</code> or <code>csv-fortran</code>).	DEFERRED
Q5	Which lab data first.	DEFERRED
Q6	Where the optimizer lives (F4), once <code>mod_fm_compare</code> exists.	DEFERRED
Q7	fortplot subplot legend. Checked 2026-09-30 against <code>lazy-fortran/fortplot</code> af5f26e: <code>figure_t%legend</code> draws all entries on top of each other in the lower-left corner of the figure and ignores location; <code>suptitle</code> does not break lines; <code>figure_t</code> has no text call. Worked around in <code>quad_plot_t</code> (Section 1.6). Reporting upstream is still open.	OPEN
Q8	Invariant or triaxial pair in the quad plot? Invariant pair (F12, 2026-09-30).	RESOLVED

1.10 Current state (review of 2026-09-30)

Item	Finding	Tag
Code	<code>src/fumat.f90</code> is a hello-world. <code>app/main.f90</code> plots $\sin^2 x$ with <code>fortplot</code> and writes PNGs. Builds and runs with the <code>fpm</code> conda env (<code>fpm</code> 0.13, <code>gfortran</code> 15.2).	–
<code>fpm.toml</code>	<code>fortplot</code> points at <code>krystophny/fortplot</code> , pinned in the build cache to a 2025 revision; <code>element-driver</code> uses <code>lazy-fortran/fortplot</code> . <code>stdlib = "*" (registry)</code> , while <code>element-driver</code> uses <code>.././stdlib</code> .	TECH DEBT
Dependency check	A scratch copy with <code>element-driver</code> as a path dependency, <code>lazy-fortran</code> <code>fortplot</code> , and <code>.././stdlib</code> built and linked. The root's <code>stdlib</code> overrode <code>critical-soil-models</code> ' git one without conflict. (That tested the direction F1 rules out, but it shows the <code>stdlib</code> and <code>fortplot</code> choices of F7 work together.)	–
Repository	<code>src/fumat.mod</code> is committed; there is no <code>.gitignore</code> . The untracked <code>text.png</code> , <code>test_100.png</code> , and <code>text.pdf</code> are outputs of <code>app/main.f90</code> .	TECH DEBT
Docs	The README names <code>environments.yml</code> (the file is <code>environment.yml</code>) and says <code>fumat</code> analyzes <code>incremental-driver</code> output. <code>environment.yml</code> installs <code>lfortran</code> (not supported,	TECH DEBT

Item	Finding	Tag
	D13). <code>.vscode/settings.json</code> uses the env <code>fpm</code> , while the README creates <code>csm</code> .	

1.11 Milestones

#	Scope
0	RESOLVED csm-tensors (F10). Repository created, code moved from <code>critical-soil-models</code> and <code>element-driver</code> , history versions added, both repositories switched, all tests passing.
1	Clean-up. Done: <code>.gitignore</code> , <code>fumat.mod</code> removed, <code>fpm.toml</code> aligned (F7), <code>test-drive</code> set up, <code>app/main.f90</code> moved to <code>example/fortplot_basics.f90</code> . To do: README and environment file.
2	Plots. Quad plot done (Section 1.6); <code>element-driver</code> depends on <code>fumat</code> (F8) and its two triaxial examples use it. Oedometer plot to do.
3	Comparison. <code>mod_fm_compare</code> : interpolation and residuals, tested with one <code>element-driver</code> run as synthetic “data”. Then decide Q6 and the <code>simulator_t</code> interface.
–	Deferred. Text files: CSV, legacy output, lab data, <code>record_t</code> readers (F9).

1.12 Change log

- 2026-09-30: notes started; decisions F1–F4 agreed; F5–F7 and the layout proposed.
- 2026-09-30: F8 (regular dependency), F9 (`element-driver` first, text files later), and F10 (shared invariants library) agreed. Sections on the shared library and on `record_t` added. Milestones renumbered.
- 2026-09-30: F5 confirmed; F10 named `csm-tensors`; F11 (no `record_t`, option A). Milestone 0 done: `csm-tensors` created and used by `critical-soil-models` and `element-driver`.
- 2026-09-30: F12 and the quad plot (`quad_plot_t`); Q7 checked and worked around; Q8 added. Milestone 1 mostly done, milestone 2 started. `element-driver` now depends on `fumat`.
- 2026-09-30: The quad plot uses the invariant pair $q = \sqrt{3J_2}, \varepsilon_q$ instead of the signed triaxial pair (F12, Q8).